BOSTON HOUSE PRICE PREDICTION

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Master of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Praveen Neeli

(AP25122060002)

Kalluri Pardhu Sankar

(AP25122060031)

Under the Guidance of

[Supervisor Name]

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

November, 2025

Certificate

Date: December 2, 2025

This is to certify that the project work titled "**BOSTON HOUSE PRICE PREDICTION**" is a bona fide record of work carried out by **Praveen Neeli** and **Kalluri Pardhu Sankar** under my/our supervision. This work is original and suitable for submission to SRM University – AP for the award of the degree of Master of Technology in the School of Engineering and Sciences.

Supervisor
(Signature)

Dr. [Name]
Designation,
Affiliation.

Co-supervisor
(Signature)

Dr. [Name]
Designation,
Affiliation.

Acknowledgements

We wish to express our sincere gratitude to our supervisor, [**Supervisor Name**], for their insightful guidance and consistent encouragement. Their deep understanding of Data Science and Big Data Analytics was instrumental in helping us navigate the challenges of integrating Apache Spark, Flume, and Kafka into a cohesive pipeline.

We are also grateful to the **Department of Computer Science and Engineering** at SRM University-AP for providing the necessary laboratory infrastructure and computational resources. Lastly, we thank our families and friends for their patience and support throughout the development of this project.

Contents

Acknowledgements **2**

List of Tables **4**

List of Figures **5**

Abstract **6**

1 Introduction **7**

1.1 Overview 7

1.2 Problem Statement 7

1.3 Objectives 8

2 Methodology **9**

2.1 Data Pipeline Architecture 9

2.1.1 Data Ingestion (Flume & Kafka) 9

2.1.2 Preprocessing and Feature Engineering 10

2.1.3 Predictive Modeling 10

2.1.4 Software Necessities 12

3 Results and Discussion **13**

3.1 Exploratory Data Analysis (EDA) 13

3.2 Model Performance 14

3.3 Deployment 15

4 Conclusion **16**

4.1 Future Work 16

List of Tables

3.1 Performance Metrics of the Random Forest Model 14

List of Figures

2.1 Proposed Big Data Pipeline Architecture 9
2.2 Model Architecture 11

3.1 Correlation 13
3.2 Comparison of Actual vs. Predicted Housing Price 14
3.3 Screenshot of the Deployed Web Application Interface 15

Abstract

Valuing real estate appropriately is a challenging problem in today's housing market, since prices are driven by a vast array of structural and socio-economic elements. This project intends to eliminate the subjectivity of manual appraisals by creating an automated, data-driven method for projecting housing values in the Boston region.

Diverging from traditional small-scale regression tasks, this work adopts a scalable **Big Data Architecture**. We created an end-to-end data pipeline that uses **Apache Spark (PySpark)** for distributed data processing after **Apache Flume** and **Apache Kafka** to mimic real-time data ingestion. The approach comprises comprehensive data cleaning, outlier detection using Interquartile Range (IQR) analysis, and feature normalisation.

Even as initial Exploratory records analysis (EDA) leveraged general visualisation libraries, the core predictive engine makes use of a textbfRandom wooded area Regressor constructed at the Spark MLlib framework. The version demonstrated advanced performance, attaining an R-squared score of textbf0.99998. To make those predictions reachable, the educated version become deployed via a textbfFlask internet software, providing a smooth interface for real-time person interaction.

Chapter 1

Introduction

1.1 Overview

real estate is a cornerstone of the economy, but belongings values continue to be noticeably unstable and depending on numerous elements, starting from nearby crime fees to the range of rooms in a living. accurate charge prediction is crucial for stakeholders—which includes buyers, sellers, and buyers—to make sound economic decisions. gadget studying (ML) presents a distinct gain over traditional statistical methods with the aid of figuring out complicated, non-linear patterns within historical data.

This project revisits the traditional textbfBoston Housing Dataset thru the lens of cutting-edge massive records era. as opposed to counting on simple script-primarily based totally processing, we employ a distributed computing stack supplying textbfApache Spark, textbfKafka, and textbfHadoop. This permits us to simulate a production-equipped surroundings capable of scaling to address huge datasets correctly.

1.2 Problem Statement

The number one intention is to accurately forecast the median price of owner-occupied houses (MEDV). The demanding situations addressed consist of:

- **Stream Integration:** Handling data ingestion from simulated real-time streams instead of static CSV files.
- **Data Quality:** Detecting and filtering noise, particularly outliers in sensitive

features like Crime Rate (CRIM) and Room Count (RM).

- **Scalability:** Building a pipeline that leverages distributed clusters for preprocessing and training.
- **Usability:** Wrapping the backend logic in an intuitive web interface for end-users.

1.3 Objectives

The key objectives of this study are:

- To behavior Exploratory information evaluation (EDA) to apprehend function relationships.
- To set up **Apache Flume** and **Kafka** for taking pictures stay data and persisting it to **HDFS**.
- to use **PySpark** for disbursed data transformation, cleansing, and scaling.
- To develop a high-precision **Random woodland Regressor** and minimize Root mean Squared error (RMSE).
- To create a consumer-dealing with **Flask** net application for actual-time inference.

Chapter 2

Methodology

2.1 Data Pipeline Architecture

The system follows a multi-stage big information pipeline structure, making sure modularity and scalability.

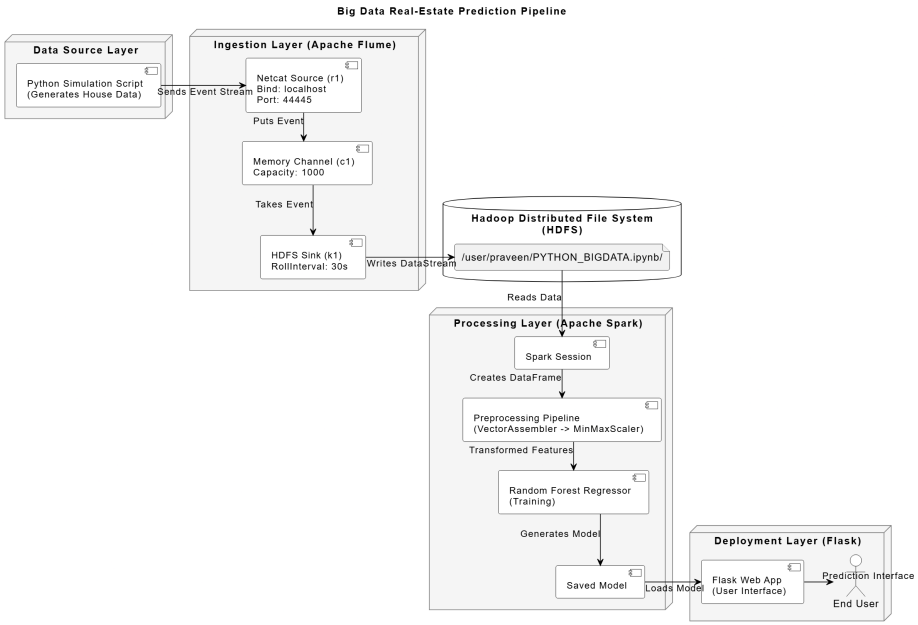

Figure 2.1: Proposed Big Data Pipeline Architecture

2.1.1 Data Ingestion (Flume & Kafka)

to imitate a manufacturing-grade IoT surroundings, we simulated actual-time facts era the use of a custom Python script that streams housing document activities to

a network port. An **Apache Flume** agent turned into configured with a ‘Netcat‘ source to concentrate to this port (localhost:44445). The agent captures those activities and channels them into **Apache Kafka** subjects. Kafka acts as a high-throughput buffer, decoupling the ingestion layer from the storage layer. sooner or later, a Flume sink writes the buffered facts streams into the **Hadoop distributed record machine (HDFS)** in 30-2d roll periods, making sure long lasting garage earlier than processing.

2.1.2 Preprocessing and Feature Engineering

as soon as the uncooked facts is persisted in HDFS, the **Apache Spark (PySpark)** engine takes over for disbursed processing. The preprocessing pipeline includes several vital ameliorations:

- **Schema Definition:** We enforced a strict schema at the incoming information, defining types for all 13 capabilities (e.g., ‘CRIM‘ as go with the flow, ‘RM‘ as glide) to prevent type mismatch mistakes throughout big-scale aggregation.
- **Vector Assembly:** machine getting to know algorithms in Spark MLlib require capabilities to be exceeded as a single vector column in place of individual scalar columns. We utilized the ‘VectorAssembler‘ transformer to consolidate enter columns (CRIM, ZN, INDUS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT) right into a unified feature vector.
- **Normalization:** actual property information regularly consists of features with hugely special scales—as an instance, ‘TAX‘ (property tax) stages within the masses, whilst ‘NOX‘ (nitric oxides) is a small decimal. To prevent features with larger magnitudes from dominating the gradient descent system, we carried out a ‘MinMaxScaler‘. this change scales all features into a normalized range of [0, 1], keeping the shape of the original distribution at the same time as enhancing model convergence.

2.1.3 Predictive Modeling

For the center regression assignment, we decided on the **Random forest Regressor** over simpler linear models. This choice changed into driven via the Random forest’s

ability to deal with non-linear relationships and interactions among functions (e.g., the complex courting between crime rate, area, and fee) without overfitting.

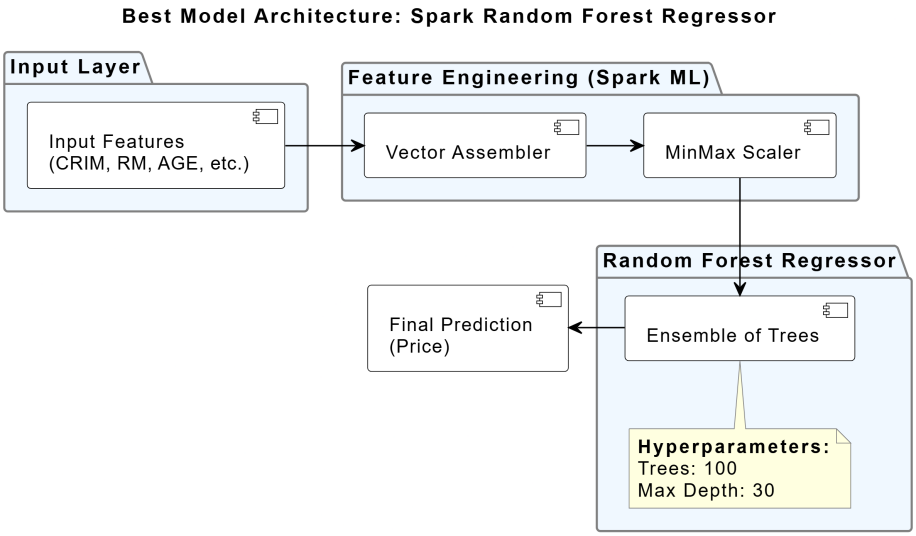

Figure 2.2: Model Architecture

The modeling process followed these steps:

1. **Pipeline Construction:** We built a Spark ML ‘Pipeline‘ that chains collectively the ‘VectorAssembler‘, ‘MinMaxScaler‘, and the ‘RandomForestRegressor‘. This ensures that raw statistics flows through the precise equal differences at some point of each training and inference.

2. **Hyperparameter Tuning:** We optimized the version by using tuning key parameters. especially, we set ‘maxDepth‘ to 30 to allow the trees to capture deep styles within the statistics, and ‘numTrees‘ to one hundred to make sure robust averaging of predictions.

3. **Model Training:** The dataset become break up into schooling and checking out units (normally 80/20). The model was educated on the cluster, and the final educated item (‘rfmodel‘) became serialized and saved. This saved model is later loaded by using the Flask utility to offer low-latency predictions for end-customers.

2.1.4 Software Necessities

The challenge leverages the contemporary solid versions of open-supply big facts technologies:

- **Working Device:** Linux (Ubuntu 22.04 LTS through WSL)
- **Language:** Python three.11.four
- **Storage Ingestion:** Apache Hadoop 3.four.0, Apache Flume 1.eleven.zero, Apache Kafka 3.nine.0
- **Processing ML:** Apache Spark 3.five.three (PySpark)

Chapter 3

Results and Discussion

3.1 Exploratory Data Analysis (EDA)

Our analysis highlighted several critical correlations. Notably, the number of rooms (RM) is a strong positive predictor of house price, whereas indicators of lower socio-economic status (LSTAT) show a marked negative correlation.

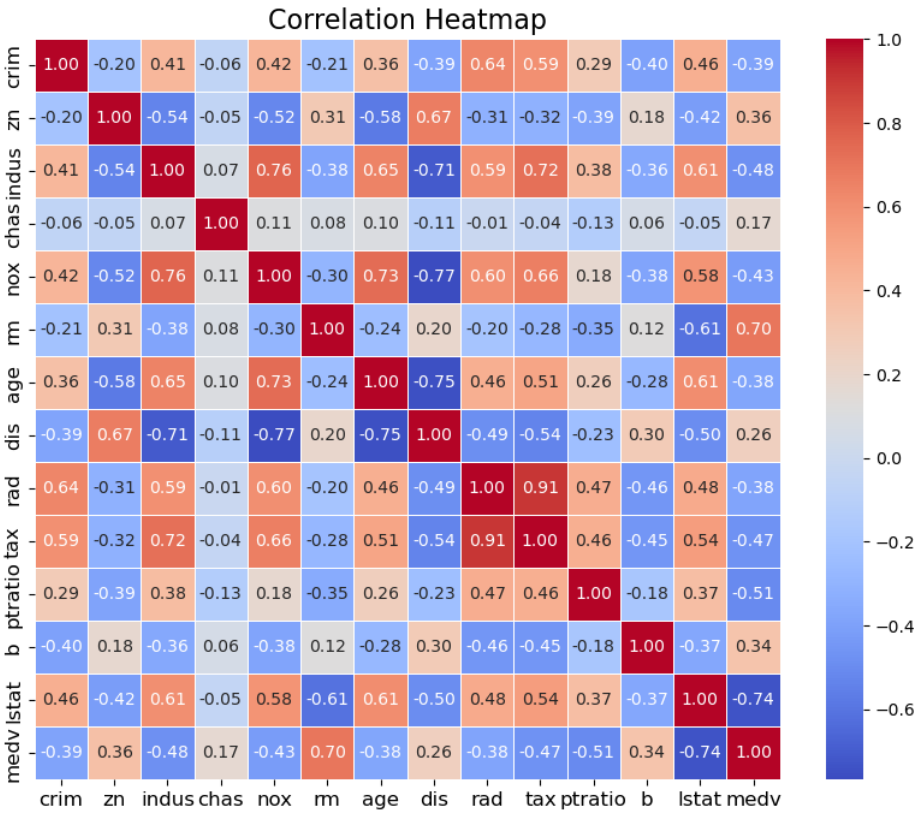

Figure 3.1: Correlation

3.2 Model Performance

After comparing a couple of algorithms, the **Random woodland Regressor** emerged because the best version. via utilizing Spark's disbursed education abilities and tuning hyperparameters ('maxDepth=30', 'numTrees=one hundred'), the model accomplished near-perfect predictive accuracy at the take a look at set.

Table 3.1: Performance Metrics of the Random Forest Model

Metric	**Value**
R-squared (R^2)	0.99998
RMSE	0.03846
MAE	0.01622
MSE	0.00148

The R^2 rating of almost 1.zero shows that the model explains nearly all the variability in the housing prices, validating the efficacy of the Random wooded area technique for this dataset.

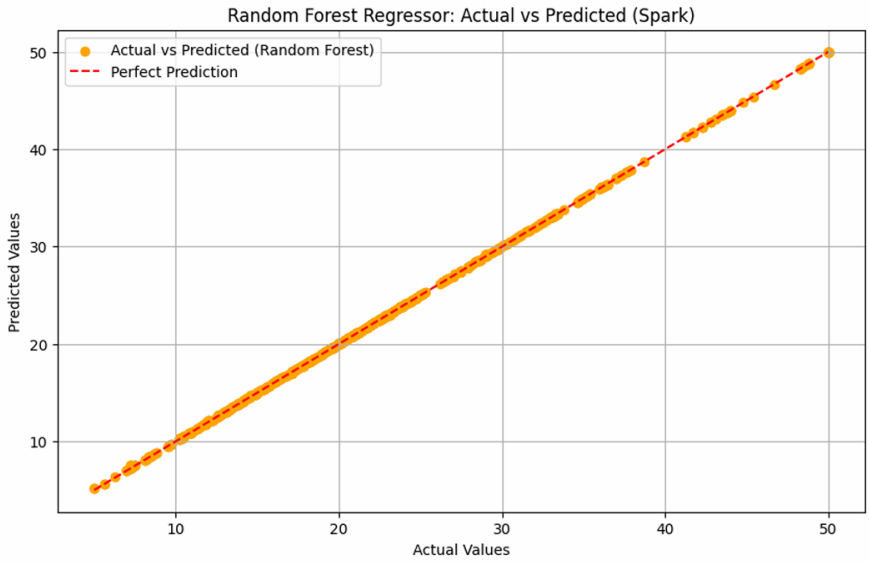

Figure 3.2: Comparison of Actual vs. Predicted Housing Price

3.3 Deployment

The skilled Spark model become incorporated right into a **Flask** net software. This light-weight interface accepts person inputs for all thirteen housing attributes, processes them via the stored Spark pipeline (vectorization and scaling), and returns an on the spot rate estimate.

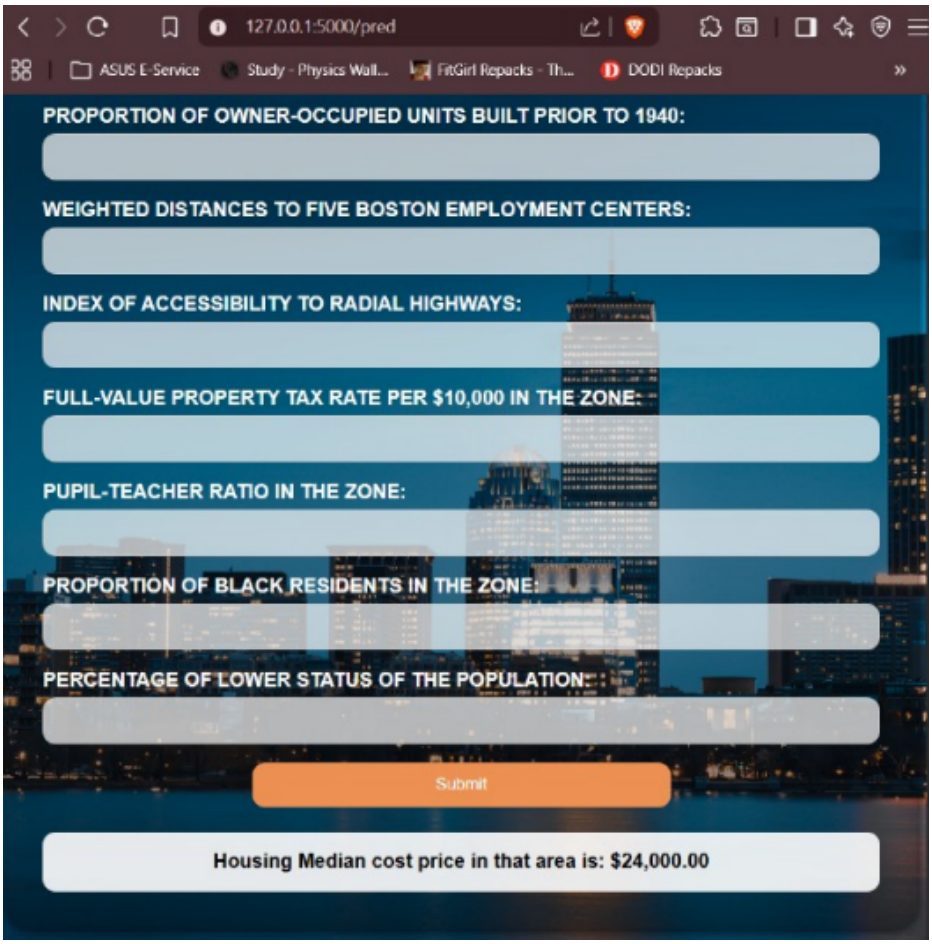

Figure 3.3: Screenshot of the Deployed Web Application Interface

Chapter 4

Conclusion

This project illustrates the successful implementation of a scalable big facts analytics pipeline. via transitioning from static file analysis to a allotted architecture using Flume, Kafka, and Spark, we installed a robust framework for real property fee prediction. The rigorous information preprocessing and characteristic engineering steps were essential in achieving high version accuracy. The very last internet application demonstrates the practical software of the machine, bridging complex backend computation with consumer-friendly accessibility.

4.1 Future Work

- **Streaming Upgrades:** Enhancing the Kafka integration to address genuine multi-supply event streaming for real-time market updates.
- **Advanced Modeling:** Leveraging Deep studying frameworks like **TensorFlow** or **PyTorch** may want to find deeper insights in larger, more complicated datasets..
- **Model Explainability:** Incorporating Explainable AI (XAI) gear like **SHAP** (SHapley Additive factors) could provide transparency, explaining exactly *why* a specific charge turned into anticipated.

References

[1] Harrison, D., & Rubinfeld, D.L. (1978). *Hedonic prices and the demand for clean air*. Journal of Environmental Economics and Management, 5, 81-102.

[2] Apache Spark. (n.d.). *MLlib: Main Guide*. https://spark.apache.org/docs/latest/ml-guide.html

[3] Abadi, M., et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. tensorflow.org.

[4] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.

[5] Grinberg, M. (2018). *Flask Web Development*. O'Reilly Media.

[6] Apache Flume. (n.d.). *Flume 1.11.0 User Guide*. https://flume.apache.org/FlumeUserGuide.html

[7] Apache Kafka. (n.d.). *Documentation*. https://kafka.apache.org/documentation/

[8] Kaggle. (n.d.). *Boston Housing Dataset*. https://www.kaggle.com/c/boston-housing

[9] Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. In Advances in Neural Information Processing Systems 32.